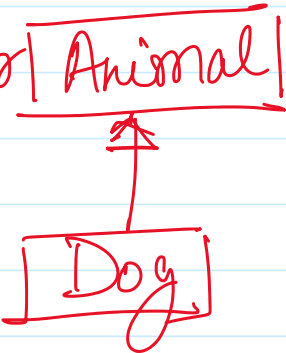


## LSP Guidelines:-

These are the guidelines to follow so that we can preserve LSP.

Guidelines are as follows:-

Note:- Broad : Parent/Ancutor Animal  
Narrow : Child Dog



### 1. Signature Rule:

(A)

```
class Parent {
    void solve (String s) {
    }
}

class Child : Public Parent {
    void solve (String s) {
    }
}
```

Signature of overridden method should be same in child class.

Why:-

B/c if child class's solve() has 'int'

and Client  $\rightarrow$  `Child *c = new Child()`  
`c  $\rightarrow$  solve(" ")` - Code breaks ~~X~~

b/c client knows solve take "string" value as parameter.

$\therefore$  Client will not able replace Child object with parent object.

This rule applies automatically by Java & C++;

Code - ... see Notes...

Also called Method Argument Rule.

Ⓑ Return Type Rule:- Organism  $\uparrow$  is a

class Parent {

Animal random() {

}

class Child : Public Parent {

Animal random() {

}

Here you could have used "Dog" But not a parent of "Animal"

Animal

$\uparrow$  is-a

Dog

Return type of child class' overridden method should be either same or narrow class

why?

Suppose we have

Parent\* P = new Parent();

Client expects Animal object in return

$\rightarrow$  Animal\* obj = P  $\rightarrow$  random()

No issue it will get the obj.

But if we used

$\rightarrow$  Dog\* obj = P  $\rightarrow$  random()

But if we used

```
Parent* p = new Child()
```

```
Animal* obj = p → random()
```

No issue :)

Now, suppose our child class looks like this

```
class Child : Public Parent {
```

```
    Dog random() {
```

```
    }
```

And client expect animal obj in return

```
Parent* p = new Child()
```

```
Animal* obj = p → random()
```

returns

Dog obj

No issue → Parent can refer to child object :)

But if child class looks like this

```
class Child : Public Parent {
```

```
    Organism random() {
```

```
    }
```

```
Parent* p = new Child()
```

```
Animal* obj = p → random
```

error

error as child can't refer to

Errors as child can't refer to  
Parent object

Also called Covariance Rule

## ③ Exception Rule.

Parent m1()



Child m1()

If m1() <sup>in Parent</sup> is throwing runtime-exception then m1() in child (overridden) should also throw same exception or its narrow exception.

Why?

Suppose Client:-

```

    try {
        P → m1();
    }
    catch (out-of-range & e)
  
```

Parent m1() → it is throwing "out-of-range" exp.  
 ↑  
Child m1() → it is throwing "logical-error" exp.

if 'P' has Parent obj then no issue  
 it will catch the exception in 'e'  
 as client is expecting "out-of-range" exception only.

But if 'P' has Child obj then ~~error~~  
 B/c Client is expecting "out-of-range" exception or its narrow exception but "logical-error" exception is the broader class of "out-of-range" so &e can't catch it.

so I can't catch it. 0 0  
↳ Error.

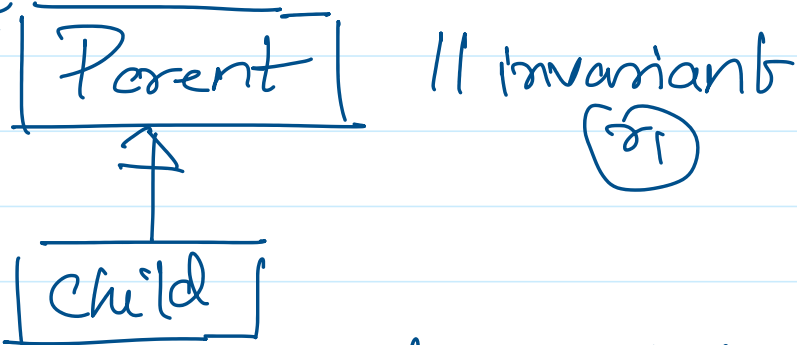
Code of "Return Type" and "Exception-  
Rule" → see notes....

## 2. Property Rule

### (A) Class Invariant

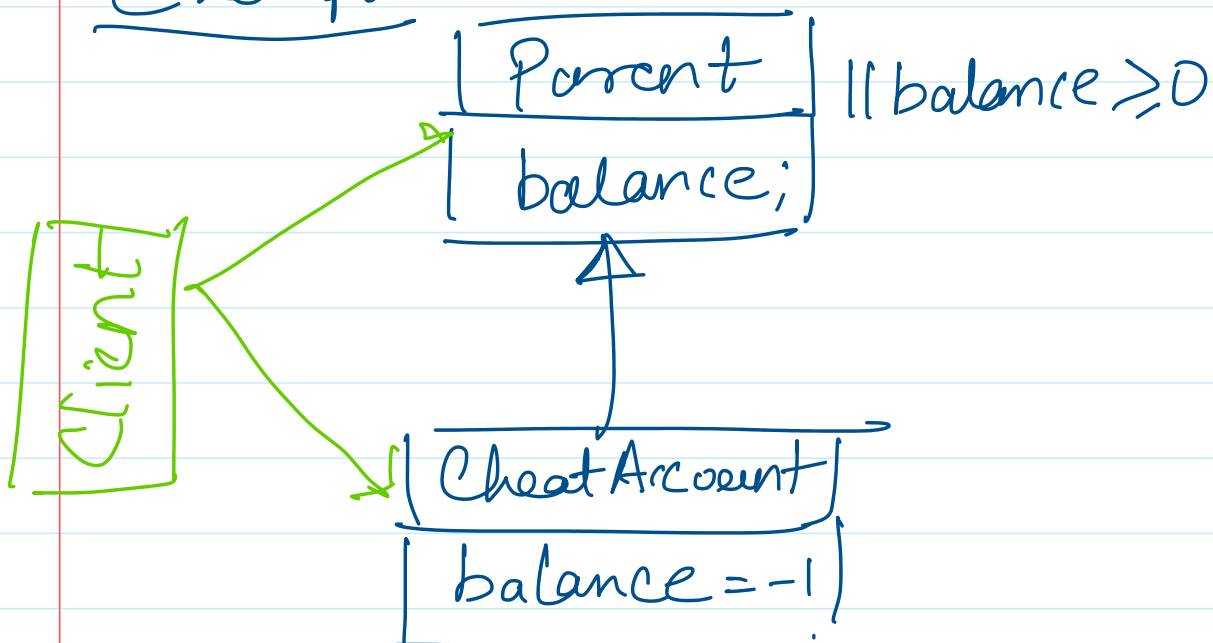
Any Rule which is always true for any class is its invariant

Example



Child class should also follow the rule  $r_1$  or strengthen it But can't make it more weak.

Example

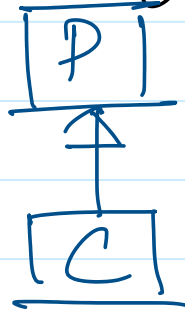


Parent class has a rule that  $\text{balance} \geq 0$   
but child class is breaking its rule  
and making it again  $-1$ .

Now client is expecting  $\text{balance} \geq 0$   
but if we pass child's Obj. then  
client's expectations will break  
So, it doesn't follow invariant Rule X

Code  $\rightarrow$  see Notes....

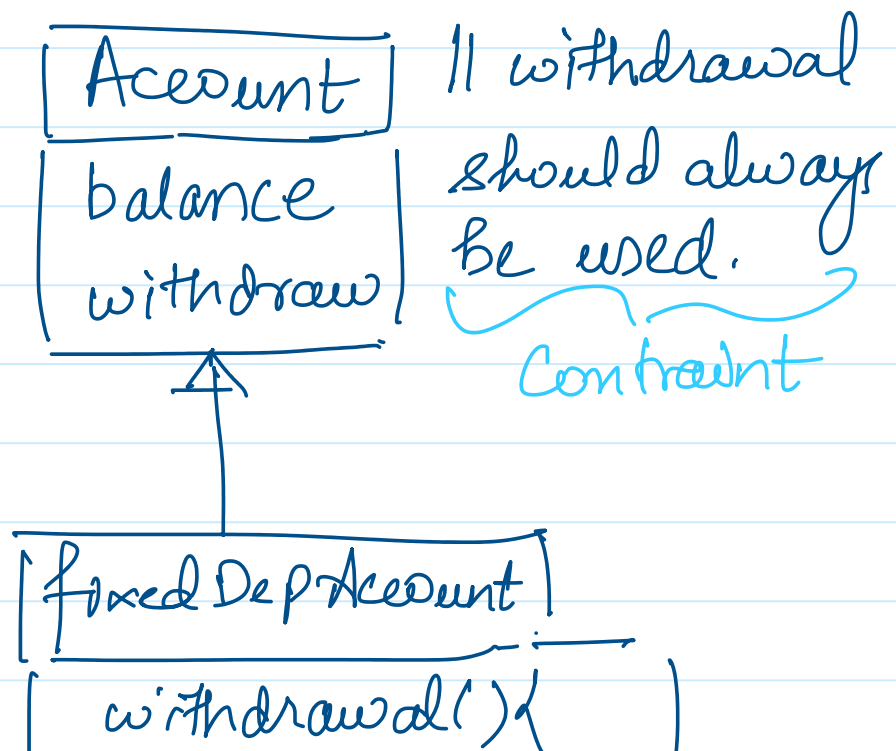
## Ⓑ History Constraint.



There is some state/history constraint that parent want to not change, it should remain as it is.

If child doesn't respect it changed the constraint then we can say that History constraint is violated.

### Example



```
withdrawal() {  
    throws exception  
}
```

So, here "fixed Dep Account" breaks the History Constraint as in this Account withdrawal is not allowed.

Code → see Notes ....

Interesting fact :-

Immutable Class :-

→ Class which can't be inherited

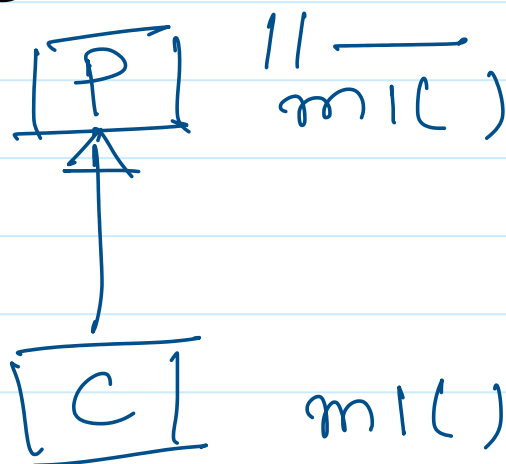
Immutable Methods :-

→ Method which can't be overridden.

If child class makes the immutable method of Parent class mutable then also it breaks LSP.

## 3. Method Rule

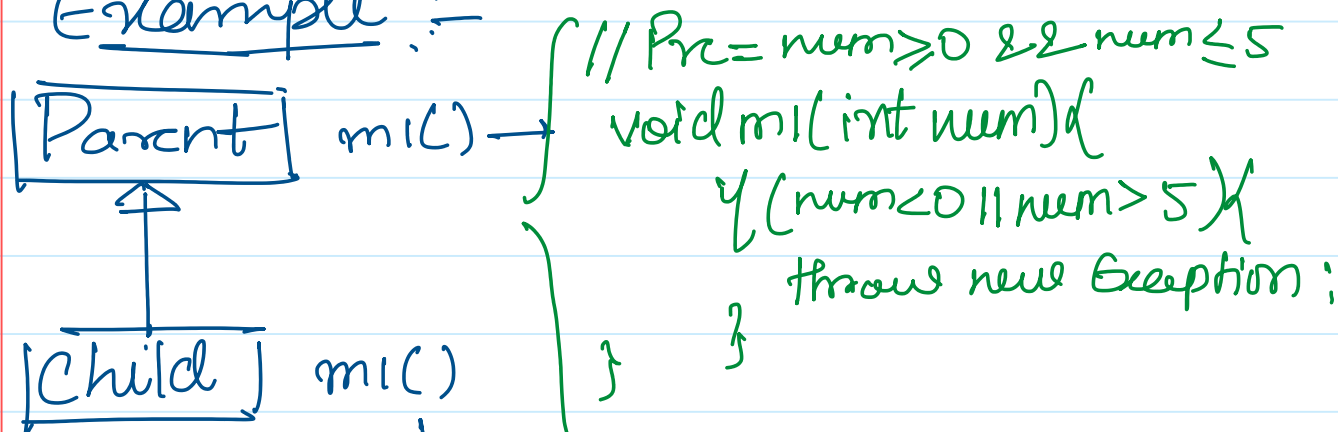
(A) Pre-condition



Suppose Parent has some function `m1()`  
 There is a condition which must be followed before `m1()`

Child class should also follow the same condition as it is or can make it weak but **can't** strengthen it by adding more constraints

Example :-



```

Child m1() {
    // Pre = num ≥ 0 & num ≤ 10
    void m1(int num) {
        if (num < 0 || num > 10) {
            throw new Except
        }
    }
}

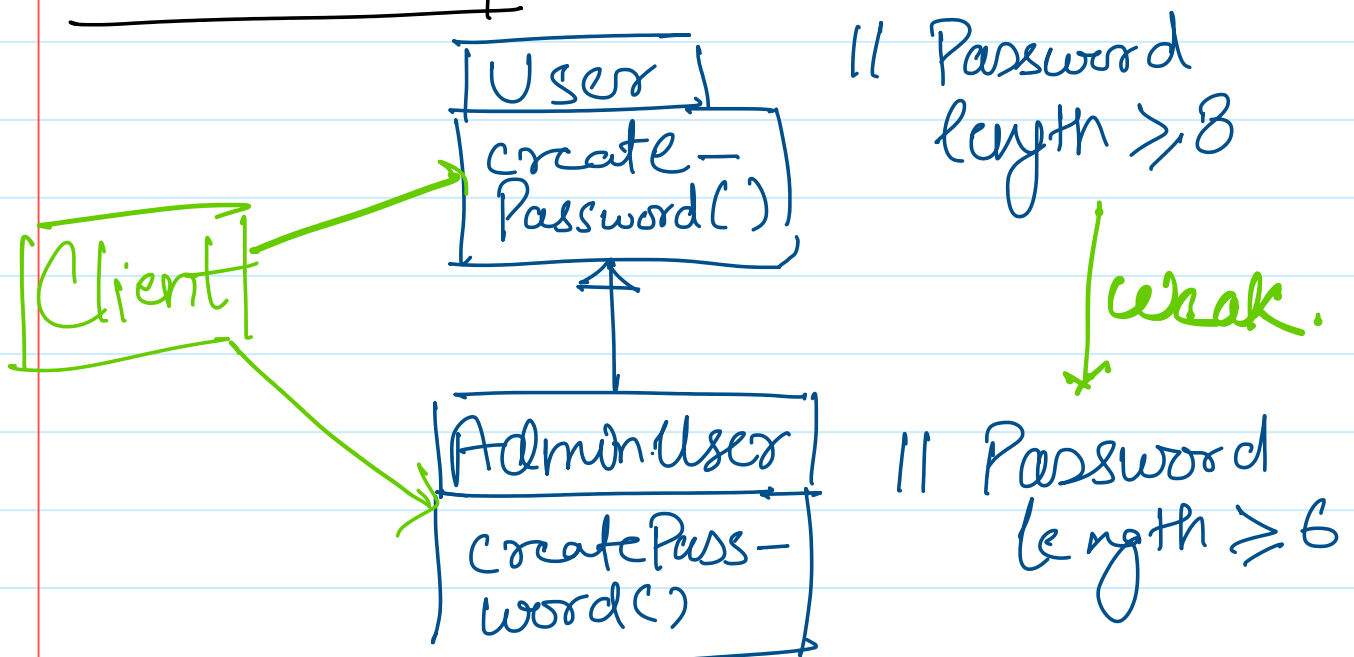
```

This will work b/c we are making the Condition more weak.

Client knows that he have to give number b/w 0 to 5 as per Parent.

So, even if Client uses child obj. then also no problem as child can also accept number from 0 to 5 and even more.

Real Example:-



No Problem Client can use either of the objects ✓

Note:- Our target is that if something is working for Parent Class object then it should also work for Child Class object.

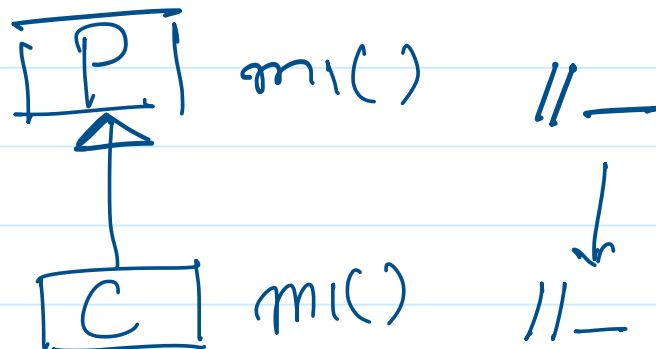
|                           | Parent | child | LSP followed |
|---------------------------|--------|-------|--------------|
| Working (✓)<br>or not (x) | ✓      | ✓     | ✓            |
|                           | x      | x     | ✓            |
|                           | ✓      | x     | x            |
|                           | x      | ✓     | x            |

This is all about LSP

Pre-Cond<sup>n</sup> Code → see Notes...

## ③ Post-Condition

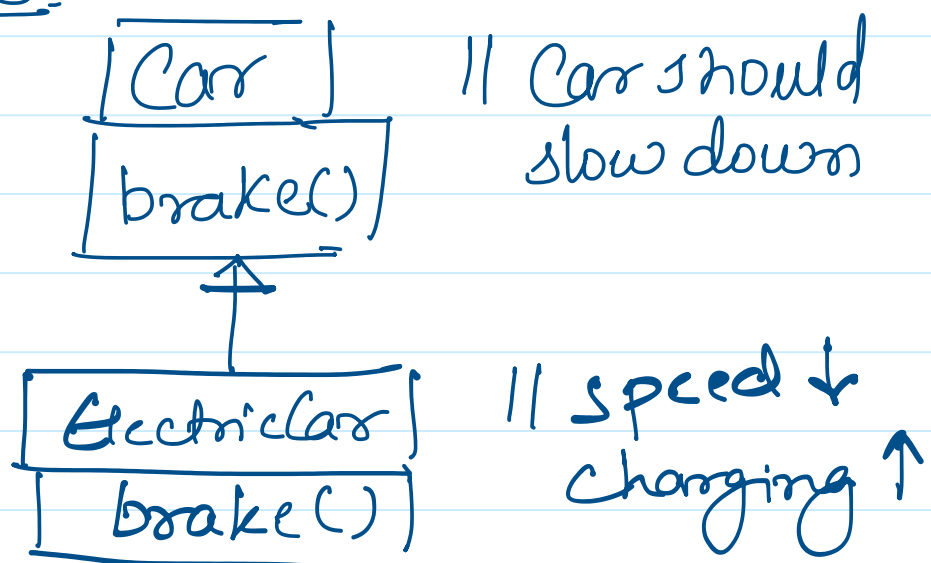
Cond<sup>n</sup> which should be true after method execution.



Child class can have Post-condition as it is or strengthen it but **not** make it weak.

↓  
Just opposite of Pre-Cond<sup>n</sup>

Example:-



Post-condition is car should slow down after brake() in Parent Class

but in Child Class we strengthen the cond<sup>n</sup> saying speed will decrease but also charging will increase.

∴ it follows LSP

if Electric car said speed will not decrease then LSP not followed.  
X

Code → see Notes....

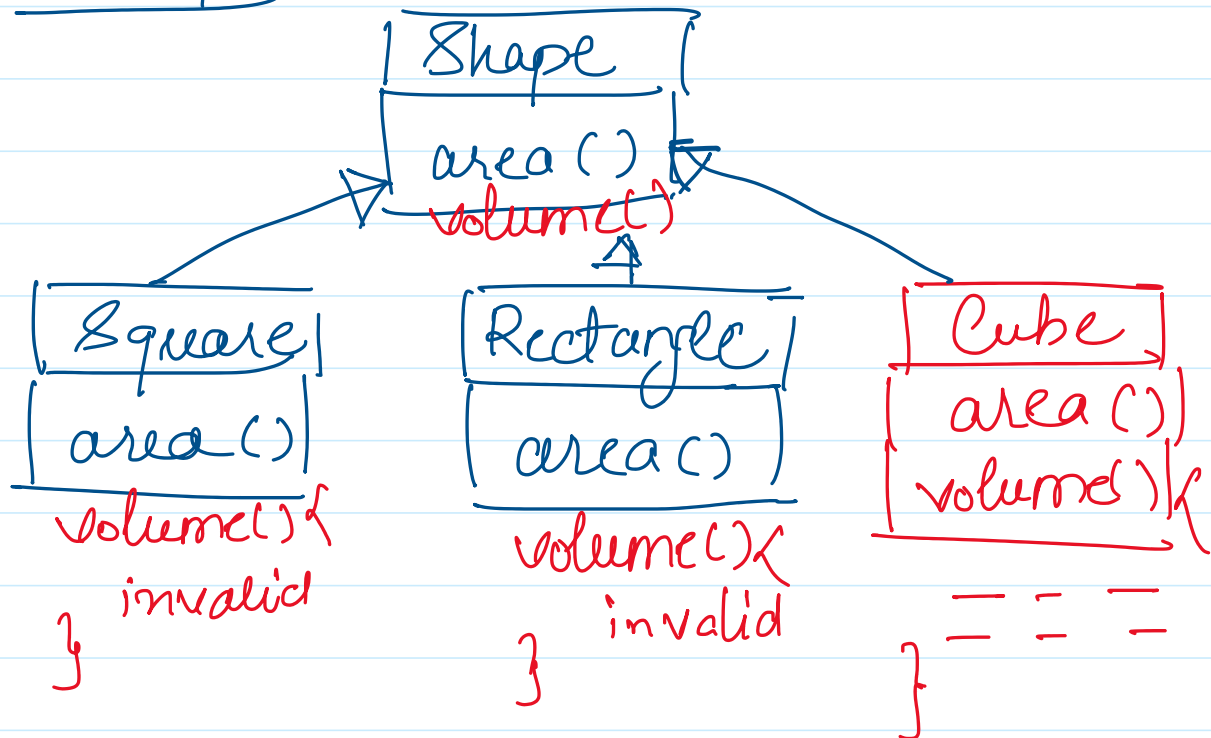
# Interface Segregation Principal

- Many Client specific interface are better than our general purpose interface.

Definition

- Client should not be forced to implement methods they don't use.

Example:- <<abstract>>



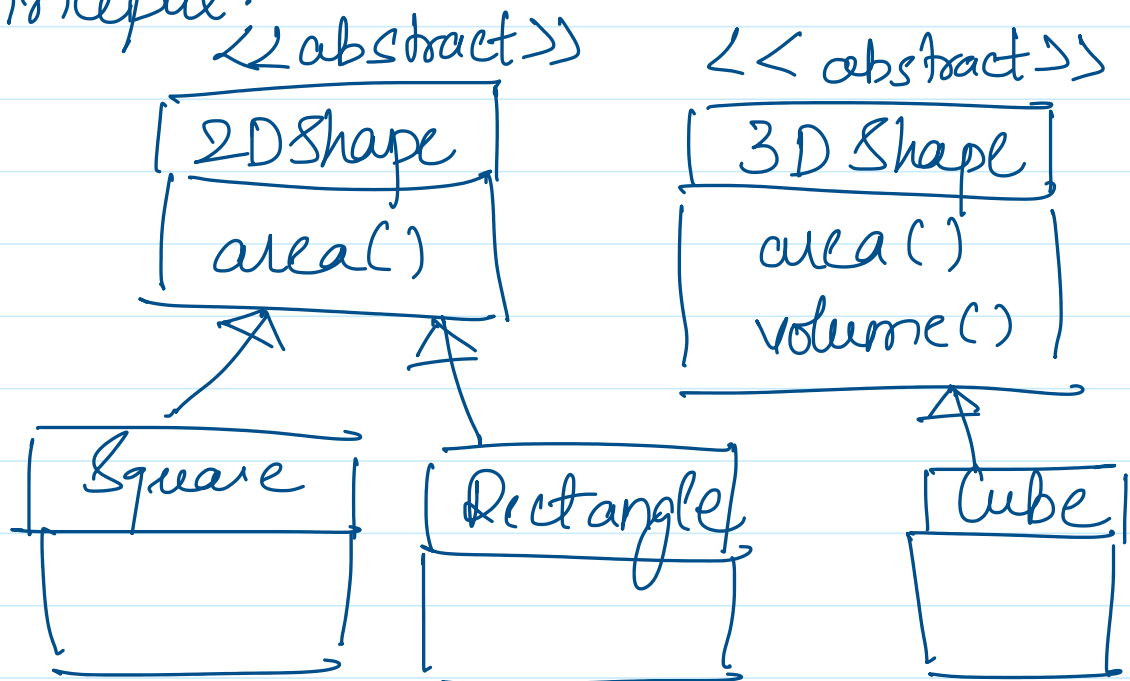
Square, Rectangle doesn't need method **volume()** but just b/c of Cube class we have to put **volume()** on Shape and b/c of it Square & Rectangle also need to override it unnecessary

also need to override it unnecessarily

Now again read the definition ↑

So what could be done - Create separate interface - 2D Shape, 3D Shape.

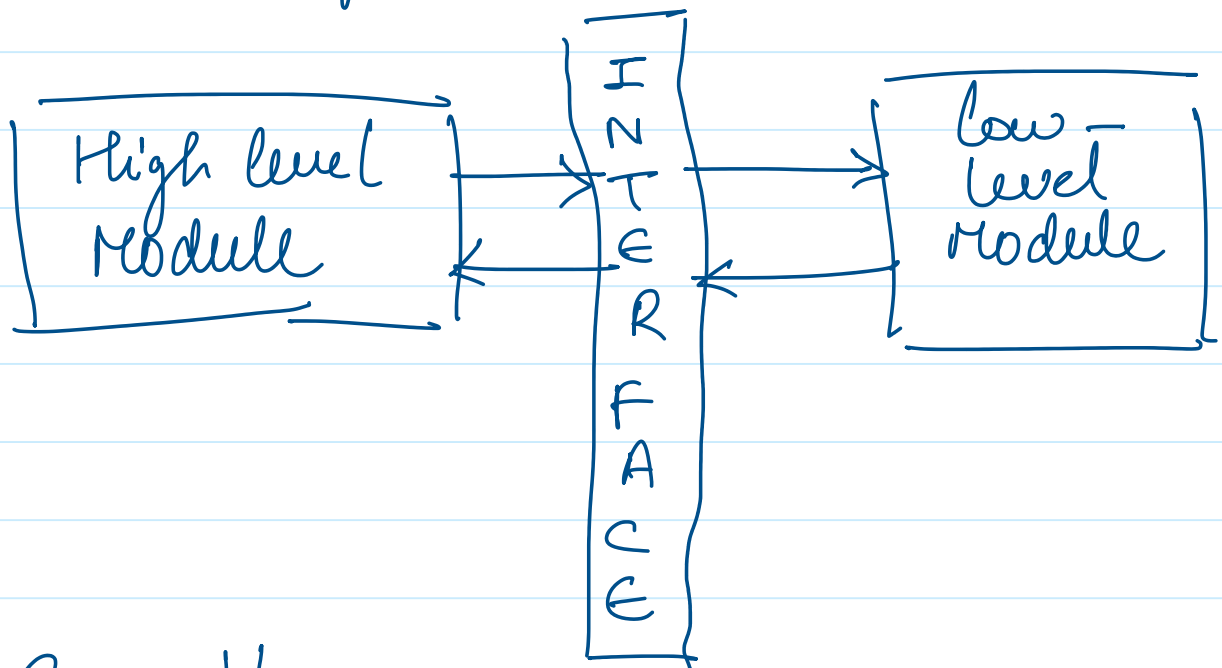
This is what we called Interface Segregation Principle.



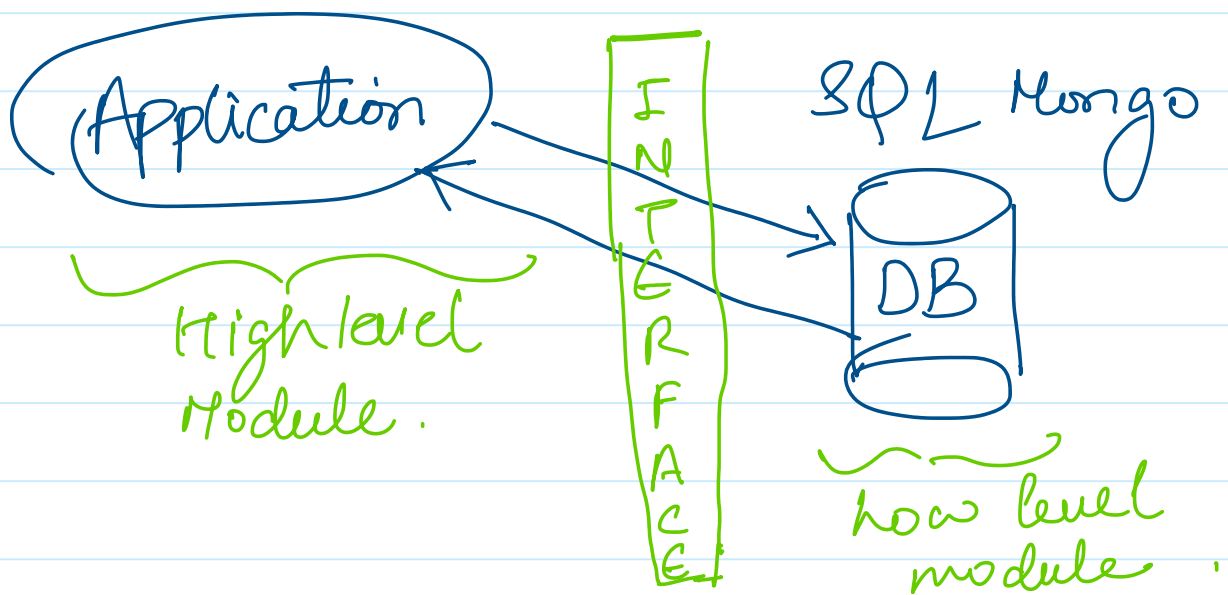
Code → see Notes...

# Dependency Inversion Principal

High level module should not depend on low level module but rather both should depend on abstraction.



## Example

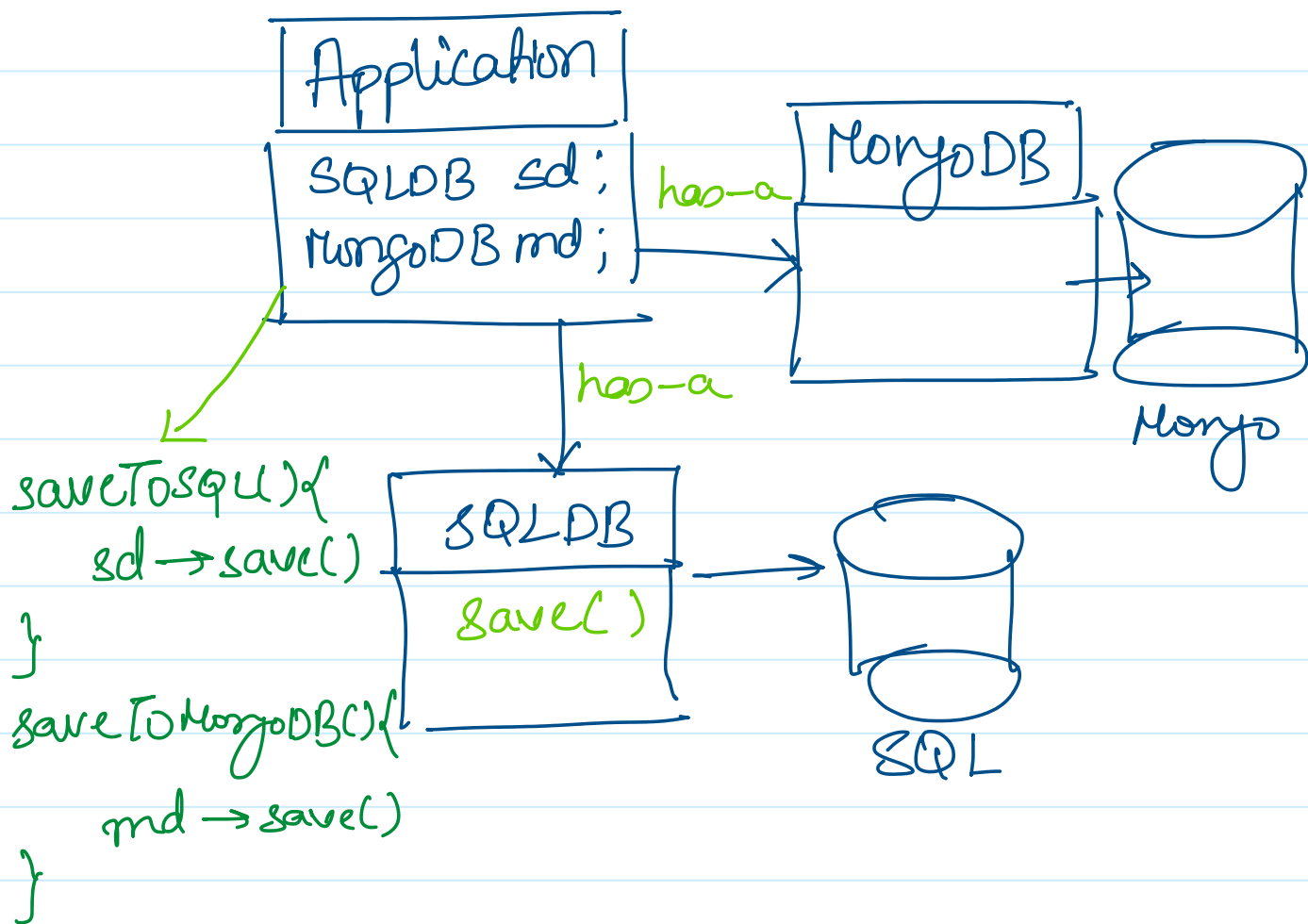


High-level Module - It deals with

High-level Module - It deals with business logic

Low-level Module - deals with system like - DB, file system etc.

Suppose

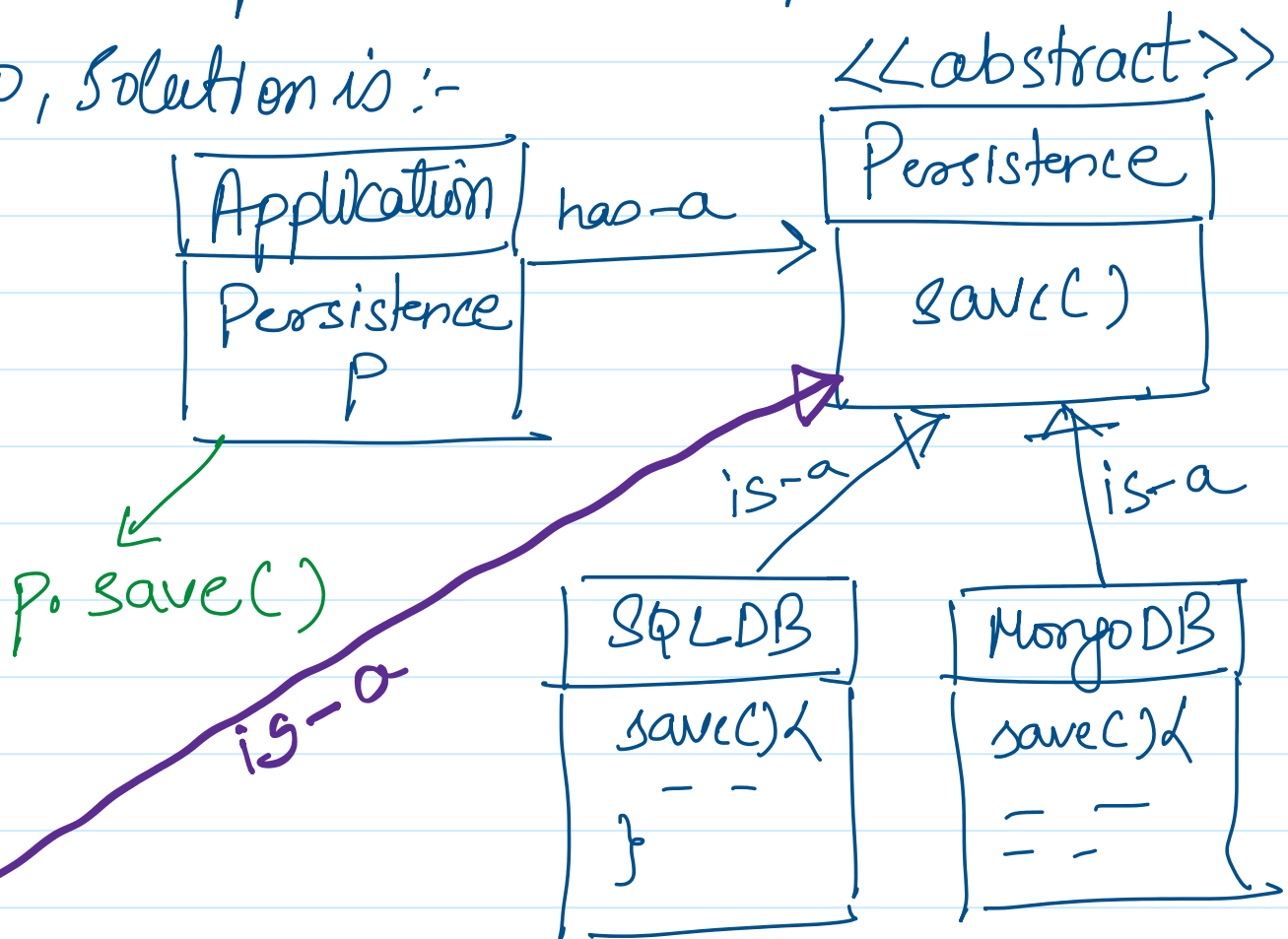


Now, suppose in future I want Cassandra DB in place of MongoDB then in that you remove 'md' object and its function as well. So you are breaking - OCP - Open-Close Principle.

So. Solution is -

<<abstract>>

So, Solution is :-



Now it depends upon 'P' to which object it is referring to.

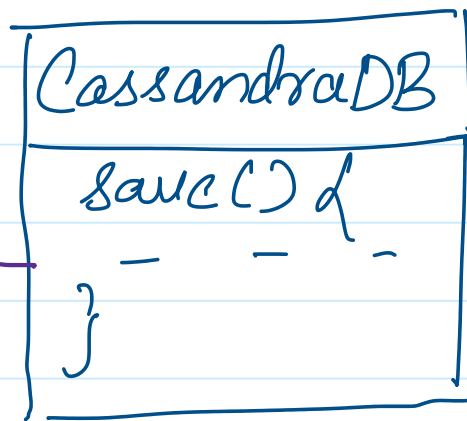
If  $P = \text{new SQLDB}()$  then save will happen in SQL DB

$P = \text{new MongoDB}()$  then save will happen in MongoDB

So, here "Persistence" acting as a abstraction b/w high-level module & low-level Module.

Now in future if Cassandra DB comes then it can be added as another implementation of the Persistence interface.

Now in future if CassandraDB comes into picture then we simply make



Code → see Notes...

Note:-

1. If OCP is the target then DIP is the solution.

## Conclusion :-

These are Principal Not laws.

So sometimes to save your logic you have break SOLID principal

So, it is kind of tradeoff b/w Business logic and SOLID Principal.

Try to maintain SOLID Principal → it will make your code more Cleaner, Expandable, Scalable.